

Математические основы алгоритмов

Экзаменационный конспект по осенней части курса

По лекциям Александра Охотина, СПбГУ, 2025–2026

Конспект организован по 22 экзаменационным билетам. Для каждого билета приведены постановки задач, ключевые инварианты, псевдокод, доказательства корректности, анализ времени и памяти, а также краткий план устного ответа.

Содержание

Обозначения и модель вычислений	5
1 Метод «разделяй и властвуй». Быстрое умножение Карацубы	6
1.1 Общая схема метода	6
1.2 Умножение в столбик и разбиение числа	6
1.3 Тождество Карацубы	6
1.4 Корректность	7
1.5 Анализ времени	7
2 Динамическое программирование: стержень и порядок умножения матриц	7
2.1 Принцип оптимальности	7
2.2 Задача о разделении стержня	8
2.3 Оптимальный порядок умножения матриц	8
3 Наибольшая общая подпоследовательность: DP и алгоритм Хиршберга	9
3.1 Постановка	9
3.2 «Народный» алгоритм	9
3.3 Алгоритм Хиршберга	10
3.4 Корректность и сложность	10
4 Сортировка вставкой, слиянием, куча и сортировка кучей	11
4.1 Сортировка вставкой	11
4.2 Сортировка слиянием	11
4.3 Двоичная куча	12
4.4 Построение кучи за линейное время	12
4.5 Heapsort	12
5 Быстрая сортировка и ожидаемое время при случайном опорном элементе	13
5.1 Разбиение	13
5.2 Худший случай	13
5.3 Ожидаемое число сравнений	13
6 Нижняя оценка сортировки. Сортировка подсчётом и поразрядная сортировка	14
6.1 Дерево решений	14
6.2 Сортировка подсчётом	14
6.3 Поразрядная сортировка	15
7 Нахождение i-го по величине элемента массива	15

7.1	Постановка и простые решения	16
7.2	Медиана медиан	16
7.3	Почему разбиение сбалансировано	16
7.4	Корректность	16
7.5	Линейное время	16
7.6	Случайный Quickselect	17
8	Поиск в графе: BFS, DFS, топологическая сортировка и КС	17
8.1	Представление графа	17
8.2	Поиск в ширину	17
8.3	Поиск в глубину	18
8.4	Топологическая сортировка	18
8.5	Компоненты сильной связности	19
8.6	Алгоритм Косараджу-Шарира	19
9	Кратчайшие пути: Беллман-Форд, Дейкстра и очередь с приоритетами	19
9.1	Релаксация	19
9.2	Беллман-Форд	20
9.3	Алгоритм Дейкстры	20
9.4	Очередь с приоритетами	21
10	Пути между всеми парами: Варшалл, Флойд-Варшалл и матричное умножение	21
10.1	Транзитивное замыкание и алгоритм Варшалла	21
10.2	Флойд-Варшалл	21
10.3	Пути и степени матрицы	22
10.4	Полукольцевая точка зрения	22
11	Быстрое умножение матриц: алгоритм Штрассена	23
11.1	Блочное представление	23
11.2	Семь произведений Штрассена	23
11.3	Сложность	23
11.4	Практические замечания	24
12	Булевы матрицы: сведение к числовым и метод четырёх русских	24
12.1	Булево произведение	24
12.2	Через числовое умножение	24
12.3	Метод четырёх русских	25
12.4	Анализ	25
12.5	Почему метод работает	25
13	Представление множеств: вектор, список, двоичное дерево поиска, АВЛ-дерево	26

13.1 Абстрактный тип «множество»	26
13.2 Массив и связный список	26
13.3 Двоичное дерево поиска	26
13.4 AVL-дерево	27
13.5 Логарифмическая высота	27
13.6 Вращения	27
14 В-деревья и понятие о красно-чёрных деревьях	28
14.1 Модель внешней памяти	28
14.2 Определение В-дерева	28
14.3 Высота	29
14.4 Поиск	29
14.5 Вставка сверху вниз	29
14.6 Удаление	29
14.7 Красно-чёрное дерево	29
15 Полиномиальное хэширование и строковые задачи	30
15.1 Полиномиальный хэш	30
15.2 Префиксные хэши	31
15.3 Алгоритм Рабина-Карпа	31
15.4 Наибольшая общая подстрока	31
15.5 Самый длинный палиндром	31
16 Алгоритм Кнута-Морриса-Пратта и конечный автомат	32
16.1 Префиксная функция	32
16.2 Поиск	32
16.3 Автоматная реализация	33
17 Префиксное дерево и алгоритм Ахо-Корасик	33
17.1 Префиксное дерево	33
17.2 Суффиксная ссылка	33
17.3 Построение автомата	34
17.4 Корректность	34
18 Сжатие Хаффмана и арифметическое кодирование	35
18.1 Префиксные коды	35
18.2 Алгоритм Хаффмана	35
18.3 Оптимальность Хаффмана	35
18.4 Арифметическое кодирование	36
18.5 Практическая реализация	36
19 Суффиксное дерево, применения и алгоритм Укконена	37
19.1 Суффиксный trie и сжатие путей	37

19.2	Линейный размер	37
19.3	Основные применения	37
19.4	Почему наивное построение квадратично	37
19.5	Неявное суффиксное дерево	38
19.6	Активная точка и суффиксные ссылки	38
19.7	Корректность	38
19.8	Линейная амортизированная сложность	39
20	Сжатие повторяющихся подстрок: LZ77 и LZ78	39
20.1	LZ77	39
20.2	Жадность	40
20.3	Скользящее окно	40
20.4	LZ78	40
20.5	Сравнение	41
21	Преобразование Берроуза-Вилера и его реализация	41
21.1	Определение	41
21.2	Почему преобразование помогает сжатию	41
21.3	Обратимость через таблицу	42
21.4	LF-перестановка	42
21.5	Быстрое прямое преобразование	42
22	Геометрические алгоритмы: ближайшая пара, выпуклая оболочка, диаметр	43
22.1	Ориентация тройки точек	43
22.2	Ближайшая пара точек	43
22.3	Почему достаточно константы соседей	43
22.4	Выпуклая оболочка и просмотр Грэма	44
22.5	Наиболее удалённая пара	44
	Сводная таблица сложностей	46
	Как готовить устный ответ	46
	Источник	47

Обозначения и модель вычислений

Пусть n обозначает размер входа, V и E — множества вершин и рёбер графа, $|w|$ — длина строки. Время работы оценивается в RAM-модели: арифметические операции над машинными словами, сравнения и обращения к ячейке памяти считаются элементарными. Если числа могут иметь существенно больше $O(\log n)$ бит, стоимость арифметики необходимо учитывать отдельно.

Запись $f(n) = O(g(n))$ означает верхнюю асимптотическую оценку, $f(n) = \Omega(g(n))$ — нижнюю, а $f(n) = \Theta(g(n))$ — совпадение оценок с точностью до постоянного множителя. Для рекурсий вида

$$T(n) = aT(n/b) + f(n)$$

полезно сравнивать $f(n)$ с $n^{\log_b a}$; в конспекте необходимые рекурсии разбираются непосредственно.

Билет 1. Метод «разделяй и властвуй». Быстрое умножение Карацубы

Ключевая идея

Разбить задачу на несколько меньших независимых задач, решить их рекурсивно и быстро объединить ответы. В алгоритме Карацубы четыре произведения половин заменяются тремя благодаря алгебраическому тождеству.

1.1 Общая схема метода

Для задачи размера n метод состоит из трёх этапов:

1. **Разделение:** представить исходную задачу через задачи меньшего размера.
2. **Рекурсивное решение:** решить подзадачи тем же алгоритмом.
3. **Слияние:** собрать ответ за время $f(n)$.

Если создаётся a подзадач размера примерно n/b , то возникает рекурсия $T(n) = aT(n/b) + f(n)$. Критически важно не только уменьшить размер подзадач, но и контролировать их суммарный объём.

1.2 Умножение в столбик и разбиение числа

Пусть два n -разрядных числа записаны в системе с основанием B . При чётном $n = 2m$ представим

$$x = x_1 B^m + x_0, \quad y = y_1 B^m + y_0,$$

где $x_0, y_0 < B^m$. Обычное раскрытие скобок даёт

$$xy = x_1 y_1 B^{2m} + (x_1 y_0 + x_0 y_1) B^m + x_0 y_0.$$

Наивная рекурсия требует четыре умножения m -разрядных чисел.

1.3 Тождество Карацубы

Вычислим только три произведения:

$$z_0 = x_0 y_0, \quad z_2 = x_1 y_1, \quad z_1 = (x_0 + x_1)(y_0 + y_1).$$

Тогда

$$x_1 y_0 + x_0 y_1 = z_1 - z_0 - z_2,$$

и

$$xy = z_2 B^{2m} + (z_1 - z_0 - z_2) B^m + z_0.$$

Сложения, вычитания и сдвиги на m разрядов требуют $O(n)$ времени.

Алгоритм: Karatsuba(x, y, n)

1. Если n мало, вернуть произведение, вычисленное обычным способом.
2. Дополнить числа ведущими нулями до одинаковой чётной длины и записать $x = x_1 B^m + x_0$, $y = y_1 B^m + y_0$.
3. Рекурсивно вычислить $z_0 = x_0 y_0$, $z_2 = x_1 y_1$, $z_1 = (x_0 + x_1)(y_0 + y_1)$.
4. Вернуть $z_2 B^{2m} + (z_1 - z_0 - z_2) B^m + z_0$.

1.4 Корректность

Доказательство / обоснование. База рекурсии корректна по определению обычного умножения. В переходе предполагаем, что три рекурсивных вызова вернули точные произведения. После подстановки

$$z_1 - z_0 - z_2 = (x_0 + x_1)(y_0 + y_1) - x_0y_0 - x_1y_1 = x_0y_1 + x_1y_0.$$

Итоговая формула совпадает с обычным раскрытием произведения xy .

1.5 Анализ времени

Получаем

$$T(n) = 3T(n/2) + O(n).$$

На уровне j дерева рекурсии имеется 3^j задач размера $n/2^j$, а суммарная нерекурсивная работа равна

$$O\left(3^j \frac{n}{2^j}\right) = O(n(3/2)^j).$$

Последний уровень доминирует. Глубина равна $\log_2 n$, поэтому

$$T(n) = \Theta(3^{\log_2 n}) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

Память при аккуратной реализации — $O(n)$ дополнительной памяти на один путь рекурсии, хотя простая реализация с многочисленными копиями массивов может использовать больше.

Важно. Алгоритм асимптотически быстрее умножения в столбик, но для малых n проигрывает из-за констант. Поэтому реальная реализация переключается на обычное умножение ниже некоторого порога.

Каркас устного ответа

Определить схему divide-and-conquer; разложить числа на старшую и младшую половины; выписать три произведения и тождество; доказать формулу; получить рекурсию $T(n) = 3T(n/2) + O(n)$ и ответ $\Theta(n^{\log_2 3})$.

Билет 2. Динамическое программирование: стержень и порядок умножения матриц

Ключевая идея

Динамическое программирование применяется, когда оптимальное решение складывается из оптимальных решений перекрывающихся подзадач. Каждое состояние вычисляется один раз и сохраняется.

2.1 Принцип оптимальности

Типичная конструкция:

1. выбрать параметр состояния, однозначно описывающий подзадачу;
2. доказать *оптимальную подструктуру*;

3. выписать рекуррентную формулу;
4. задать порядок заполнения таблицы;
5. при необходимости хранить выбор, чтобы восстановить решение.

Мемоизация вычисляет состояния сверху вниз; табличный метод — снизу вверх.

2.2 Задача о разделении стержня

Дан стержень длины n и цены p_i за кусок длины i . Разрешено разрезать стержень на целые части. Требуется максимизировать выручку.

Пусть r_j — максимальная выручка для длины j . Если первый кусок имеет длину i , остаток длины $j - i$ должен быть разрезан оптимально, иначе весь план можно улучшить. Поэтому

$$r_0 = 0, \quad r_j = \max_{1 \leq i \leq j} \{p_i + r_{j-i}\}.$$

Алгоритм: Rod-Cutting

1. Положить $r_0 = 0$.
2. Для $j = 1, \dots, n$ вычислить $r_j = \max_{1 \leq i \leq j} (p_i + r_{j-i})$.
3. Вместе с r_j сохранить s_j — длину первого куска, на которой достигнут максимум.
4. Для восстановления ответа последовательно выводить $s_n, s_{n-s_n}, \dots$

Доказательство / обоснование. Рассмотрим оптимальное разбиение стержня длины j и его первый кусок длины i . Если остаток разрезан не оптимально, заменим его на более выгодное разбиение и увеличим общую выручку — противоречие. Значит, оптимум имеет вид $p_i + r_{j-i}$ для некоторого i , а максимум по всем i даёт точный ответ.

Сложность. Время $\Theta(n^2)$, память $\Theta(n)$. Наивная рекурсия без запоминания экспоненциальна, поскольку многократно решает одинаковые подзадачи.

2.3 Оптимальный порядок умножения матриц

Дана цепочка

$$A_1 A_2 \cdots A_n,$$

где A_i имеет размер $d_{i-1} \times d_i$. Порядок скобок не меняет результат, но меняет число скалярных умножений. Умножение матриц размеров $a \times b$ и $b \times c$ стоит abc операций.

Пусть $m[i, j]$ — минимальная стоимость вычисления $A_i \cdots A_j$. Последнее умножение делит цепочку между A_k и A_{k+1} :

$$m[i, i] = 0,$$

$$m[i, j] = \min_{i \leq k < j} (m[i, k] + m[k+1, j] + d_{i-1} d_k d_j).$$

Последний член — стоимость перемножения уже вычисленных матриц размеров $d_{i-1} \times d_k$ и $d_k \times d_j$.

Алгоритм: Matrix-Chain-Order

1. Инициализировать $m[i, i] = 0$.
2. Для длины интервала $\ell = 2, \dots, n$ перебрать все i , положить $j = i + \ell - 1$.
3. Перебрать $k = i, \dots, j - 1$ и выбрать минимум рекуррентной формулы.
4. Сохранить оптимальный разрез $s[i, j] = k$; рекурсивно восстановить скобки по таблице s .

Доказательство / обоснование. В любом полном порядке скобок существует последнее умножение, которое соединяет две подцепочки $A_i \cdots A_k$ и $A_{k+1} \cdots A_j$. Обе подцепочки в оптимальном решении обязаны быть вычислены оптимально: иначе замена одной из них уменьшила бы общую стоимость. Перебор всех k поэтому исчерпывает все возможные оптимальные решения.

Сложность. Состояний $\Theta(n^2)$, для каждого перебирается $O(n)$ разрезов: время $\Theta(n^3)$, память $\Theta(n^2)$.

Каркас устного ответа

Сформулировать принцип оптимальности; для стержня выписать $r_j = \max_i(p_i + r_{j-i})$; для цепочки матриц определить $m[i, j]$ и последнее умножение; доказать оптимальную подструктуру; назвать сложности $O(n^2)$ и $O(n^3)$.

Билет 3. Наибольшая общая подпоследовательность: DP и алгоритм Хиршберга

Ключевая идея

Обычный алгоритм LCS строит таблицу mn и восстанавливает ответ обратным проходом. Хиршберг сохраняет время $O(mn)$, но уменьшает память до $O(\min(m, n))$, сочетая divide-and-conquer с двумя линейными по памяти проходами динамического программирования.

3.1 Постановка

Подпоследовательность получается удалением некоторых символов без изменения порядка оставшихся. Для строк

$$u = a_1 \cdots a_m, \quad v = b_1 \cdots b_n$$

нужно найти общую подпоследовательность максимальной длины.

3.2 «Народный» алгоритм

Пусть $L[i, j]$ — длина LCS префиксов $a_1 \cdots a_i$ и $b_1 \cdots b_j$. Тогда

$$L[0, j] = L[i, 0] = 0,$$

$$L[i, j] = \begin{cases} L[i-1, j-1] + 1, & a_i = b_j, \\ \max\{L[i-1, j], L[i, j-1]\}, & a_i \neq b_j. \end{cases}$$

Доказательство / обоснование. Если $a_i = b_j$, существует оптимальная LCS, заканчивающаяся этим символом; удаление последнего символа сводит задачу к префиксам $(i-1, j-1)$. Если символы различны, они не могут оба быть последним символом одной и той же общей подпоследовательности, поэтому хотя бы один из них не используется, и остаётся максимум двух меньших задач.

Для восстановления ответа идём из (m, n) к $(0, 0)$. При равных символах добавляем символ и идём по диагонали; иначе идём в соседнюю ячейку с не меньшим значением. Время и память равны $\Theta(mn)$. Если нужна только длина, достаточно хранить две строки таблицы, то есть $O(n)$ памяти.

3.3 Алгоритм Хиршберга

Предположим $m \geq 2$ и разделим

$$u = u'u'', \quad |u'| = \lfloor m/2 \rfloor.$$

Любое оптимальное совмещение пересекает границу между u' и u'' в некоторой позиции j строки v :

$$v = v'v'', \quad |v'| = j.$$

Нужно выбрать j , максимизирующее

$$F[j] + G[j],$$

где $F[j] = \text{LCS}(u', v[1..j])$, а $G[j] = \text{LCS}(u'', v[j+1..n])$.

Массив F вычисляется обычным DP, но хранятся только две строки. Массив G вычисляется симметрично справа налево. После выбора максимального j рекурсивно решаются две задачи (u', v') и (u'', v'') .

Алгоритм: Hirschberg(u, v)

1. Если одна строка пуста, вернуть пустую строку. Если $|u| = 1$, проверить наличие её символа в v .
2. Разбить $u = u'u''$ пополам.
3. Вычислить массивы $F[j] = \text{LCS}(u', v[1..j])$ и $G[j] = \text{LCS}(u'', v[j+1..n])$ в линейной памяти.
4. Выбрать $j \in \arg \max(F[j] + G[j])$.
5. Вернуть конкатенацию $\text{Hirschberg}(u', v[1..j])$ и $\text{Hirschberg}(u'', v[j+1..n])$.

3.4 Корректность и сложность

Доказательство / обоснование. Рассмотрим конкретное оптимальное совмещение. В момент перехода от символов u' к символам u'' оно разделяет используемые позиции v после некоторой позиции j . Длина левой части не превосходит $F[j]$, правой — $G[j]$. Следовательно, длина оптимума не превосходит $\max_j (F[j] + G[j])$. Обратно, для каждого j конкатенация оптимальных подпоследовательностей двух частей является общей подпоследовательностью исходных строк, поэтому максимум достижим.

На каждом уровне рекурсии суммарная площадь DP-таблиц не превосходит mn , а размеры первой строки в рекурсивных задачах уменьшаются вдвое. Точная индукция даёт $T(m, n) = O(mn)$. Одновременно хранятся два массива длины n и стек глубины $O(\log m)$, поэтому память $O(n)$; меняя строки местами, получаем $O(\min(m, n))$.

Важно. Хиршберг экономит память, но не время. Его ключевой шаг — не произвольное деление второй строки, а вычисление оптимальной точки разреза через два DP-прохода.

Каркас устного ответа

Дать рекурсию обычного LCS и способ восстановления. Затем разделить первую строку, определить массивы прямых и обратных LCS, выбрать максимум их суммы, доказать корректность через пересечение оптимального совмещения с границей и назвать $O(mn)$ времени, $O(\min(m, n))$ памяти.

Билет 4. Сортировка вставкой, слиянием, куча и сортировка кучей

Ключевая идея

Вставки поддерживают отсортированный префикс; merge sort рекурсивно сортирует половины и линейно сливает; куча поддерживает экстремальный элемент в корне и позволяет получить сортировку за $O(n \log n)$.

4.1 Сортировка вставкой

На шаге i префикс $a_1, \dots, a_{i-1}$ уже отсортирован. Элемент a_i сдвигается влево, пока не займёт правильное место.

Инвариант. Перед началом итерации i первые $i - 1$ элементов содержат те же значения, что и исходный префикс, и расположены по неубыванию.

После вставки инвариант сохраняется. В худшем случае каждый элемент проходит через весь префикс:

$$1 + 2 + \dots + (n - 1) = \Theta(n^2).$$

В лучшем случае массив уже отсортирован, и время $\Theta(n)$. Сортировка устойчива и работает на месте.

4.2 Сортировка слиянием

Массив делится пополам, обе половины сортируются рекурсивно, затем два отсортированных массива сливаются двумя указателями.

Алгоритм: Merge

1. Держать указатели i, j на первые неиспользованные элементы двух половин.
2. Сравнить текущие элементы и перенести меньший в выходной массив.
3. Когда одна половина закончилась, дописать остаток другой.

Доказательство / обоснование. На каждом шаге минимальный элемент среди ещё не выведенных обязан находиться в начале одной из половин. Алгоритм выбирает меньший из этих двух кандидатов, поэтому выходной префикс остаётся отсортированным и содержит именно наименьшие уже обработанные элементы.

Рекурсия $T(n) = 2T(n/2) + \Theta(n)$ даёт $\Theta(n \log n)$. Память стандартной реализации $\Theta(n)$. Сортировка устойчива.

4.3 Двоичная куча

Min-heap — почти полное двоичное дерево, хранимое в массиве. Для индексации с единицы:

$$\text{parent}(i) = \lfloor i/2 \rfloor, \quad \text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1.$$

Условие кучи: $a_{\text{parent}(i)} \leq a_i$. Поэтому минимум находится в корне.

Основные операции:

- min — $O(1)$;
- вставка — добавить лист и поднимать вверх, $O(\log n)$;
- extract_min — заменить корень последним элементом и опускать вниз, $O(\log n)$;
- decrease_key — уменьшить ключ и поднимать вверх, $O(\log n)$.

Процедура *heapify* опускает нарушающий условие элемент, каждый раз меняя его с меньшим потомком. Если оба поддерева уже кучи, после окончания всё поддерево становится кучей.

4.4 Построение кучи за линейное время

Запускаем *heapify* для внутренних вершин в порядке $\lfloor n/2 \rfloor, \dots, 1$. Грубая оценка $O(n \log n)$ неточна. Вершин высоты h не более $n/2^{h+1}$, поэтому

$$\sum_{h \geq 0} O\left(\frac{n}{2^{h+1}} h\right) = O(n) \sum_{h \geq 0} \frac{h}{2^h} = O(n).$$

4.5 Heapsort

Для сортировки по возрастанию удобно построить max-heap. Затем многократно меняем корень с последним элементом активной части, уменьшаем размер кучи и выполняем *heapify* в корне. Извлечённые максимумы накапливаются справа.

Сложность. Построение $O(n)$, затем $n-1$ извлечений по $O(\log n)$: $O(n \log n)$. Дополнительная память $O(1)$, сортировка неустойчива.

Алгоритм	Время, худшее	Память	Устойчивость
Вставки	$O(n^2)$	$O(1)$	да
Слияние	$O(n \log n)$	$O(n)$	да
Куча	$O(n \log n)$	$O(1)$	нет

Каркас устного ответа

Для вставок назвать инвариант отсортированного префикса. Для merge sort описать слияние и рекурсию. Определить массивное представление кучи, *heapify* и операции. Отдельно обосновать линейное построение кучи суммой по высотам и получить heapsort $O(n \log n)$.

Билет 5. Быстрая сортировка и ожидаемое время при случайном опорном элементе

Ключевая идея

Quicksort ставит опорный элемент на окончательное место и рекурсивно сортирует две части. Случайный выбор опорного не исключает плохие разбиения, но обеспечивает ожидаемое время $O(n \log n)$.

5.1 Разбиение

Выбирается опорный элемент x . Массив переставляется так, чтобы слева находились элементы $\leq x$, справа — элементы $\geq x$, а сам x оказался между ними. В варианте Ломута опорный обычно переносится в конец; вариант Хоара использует два указателя, движущихся навстречу.

Алгоритм: Randomized-Quicksort

1. Если размер части не превосходит единицы, завершить.
2. Равновероятно выбрать один из её элементов как опорный.
3. Выполнить partition за линейное время.
4. Рекурсивно отсортировать левую и правую части.

Доказательство / обоснование. После partition каждый элемент левой части не превосходит опорный, а каждый элемент правой не меньше. По индукции рекурсивные вызовы сортируют обе части. Опорный уже находится между ними, поэтому весь массив отсортирован.

5.2 Худший случай

Если опорный каждый раз минимален или максимален, то

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2).$$

Случайная перестановка входа или случайный опорный делают поведение независимым от исходного порядка, но худший случай как событие остаётся возможным.

5.3 Ожидаемое число сравнений

Пусть элементы после сортировки имеют ранги $1 < \dots < n$. Для пары $i < j$ введём индикатор X_{ij} события, что эти два элемента когда-либо сравниваются. Они сравниваются тогда и только тогда, когда первым выбранным опорным среди элементов рангов $i, i+1, \dots, j$ окажется один из концов i или j . Поэтому

$$\Pr[X_{ij} = 1] = \frac{2}{j-i+1}.$$

Общее число сравнений $X = \sum_{i < j} X_{ij}$, и по линейности ожидания

$$\mathbb{E}X = \sum_{i < j} \frac{2}{j-i+1} = 2 \sum_{d=1}^{n-1} \frac{n-d}{d+1} = O(n \log n).$$

Более точно, $\mathbb{E}X = 2(n+1)H_n - 4n$, где $H_n = \Theta(\log n)$.

Каждое сравнение сопровождается константной работой, поэтому ожидаемое время $O(n \log n)$. Ожидаемая глубина стека также $O(\log n)$, но без специальных мер худшая глубина равна n . Для гарантии $O(\log n)$ памяти рекурсивно обрабатывают меньшую часть, а по большей продолжают цикл.

Важно. Нельзя доказывать среднее время фразой «в среднем опорный делит массив пополам». Это неверно для отдельного шага. Корректный анализ использует либо рекурсию по всем возможным рангам опорного, либо индикаторы сравнений.

Каркас устного ответа

Описать partition и индуктивную корректность; показать квадратичный худший случай; для случайного выбора ввести пары рангов, вероятность $2/(j-i+1)$ и просуммировать до $O(n \log n)$.

Билет 6. Нижняя оценка сортировки. Сортировка подсчётом и поразрядная сортировка

Ключевая идея

Любая сортировка, получающая информацию только из сравнений, требует $\Omega(n \log n)$ сравнений. Линейные алгоритмы возможны, если использовать структуру ключей, например ограниченный диапазон цифр.

6.1 Дерево решений

Рассмотрим сортировку n попарно различных элементов, основанную только на сравнениях. Её выполнение представляется бинарным деревом: внутренняя вершина — сравнение, две ветви — два исхода. Каждая из $n!$ перестановок должна приводить к отдельному листу, иначе алгоритм не сможет выдать правильный порядок для двух разных входов.

Бинарное дерево высоты h имеет не более 2^h листьев, поэтому

$$2^h \geq n!, \quad h \geq \log_2(n!).$$

По формуле Стирлинга или оценке $(n/2)^{n/2} \leq n!$ получаем

$$\log_2(n!) = \Theta(n \log n).$$

Значит, в худшем случае требуется $\Omega(n \log n)$ сравнений. Merge sort и heapsort асимптотически оптимальны в этой модели.

6.2 Сортировка подсчётом

Пусть ключи — целые числа $0, \dots, k-1$. Создаётся массив счётчиков $C[0..k-1]$.

Алгоритм: Counting-Sort

1. Обнулить C и посчитать частоты ключей.
2. Для простой неустойчивой версии вывести каждый ключ x ровно $C[x]$ раз.
3. Для устойчивой версии заменить частоты префиксными суммами: $C[x]$ станет числом элементов с ключом $\leq x$.
4. Просмотреть вход справа налево, помещая элемент с ключом x в позицию $C[x]$ и уменьшая $C[x]$.

Сложность. Время $\Theta(n + k)$, память $\Theta(n + k)$ для устойчивой версии. Если $k = O(n)$, время линейное.

6.3 Поразрядная сортировка

Пусть каждый ключ состоит из d цифр в системе с основанием k . LSD-radix sort устойчиво сортирует сначала по младшей цифре, затем по следующей и так далее.

Доказательство / обоснование. После обработки первых t разрядов элементы упорядочены по t младшим разрядам. Устойчивая сортировка по разряду $t + 1$ группирует элементы по новой цифре, но внутри каждой группы сохраняет порядок по уже обработанным младшим цифрам. Поэтому инвариант выполняется для $t + 1$.

Если каждый проход выполнен counting sort, то

$$T = \Theta(d(n + k)).$$

Например, для w -битных чисел можно брать блоки по r бит: $d = \lceil w/r \rceil$, $k = 2^r$. Выбор $2^r = \Theta(n)$ даёт $O(wn/\log n)$; при $w = O(\log n) - O(n)$.

Важно. Линейность не противоречит нижней оценке: counting/radix sort используют не только сравнения, а прямую адресацию по значениям цифр. Устойчивость внутренней сортировки для LSD-метода обязательна.

Каркас устного ответа

Построить дерево решений с $n!$ листьями и вывести $\log(n!) = \Omega(n \log n)$. Затем описать частоты и префиксные суммы counting sort, после чего доказать radix sort инвариантом обработанных разрядов.

Билет 7. Нахождение i -го по величине элемента массива**Ключевая идея**

Порядковую статистику можно найти без полной сортировки. Детерминированный алгоритм BFPRT выбирает медиану медиан как гарантированно некрайний опорный элемент и работает за $O(n)$ в худшем случае.

7.1 Постановка и простые решения

i -я порядковая статистика — элемент, который стоял бы на позиции i после сортировки. Полная сортировка требует $O(n \log n)$. Для минимума или максимума достаточно $O(n)$, и оказывается, что линейное время возможно для любого i .

7.2 Медиана медиан

Разобьём элементы на группы по пять, отсортируем каждую группу за константное время и выпишем её медиану. Рекурсивно найдём медиану y массива медиан. Затем разделим исходный массив на элементы $< y$, $= y$, $> y$ и продолжим только в той части, где находится искомый ранг.

Алгоритм: $\text{Select}(A, i)$

1. Если $|A|$ мало, отсортировать A и вернуть элемент ранга i .
2. Разбить A на группы по пять; в каждой найти медиану.
3. Рекурсивно найти медиану y массива медиан.
4. За $O(n)$ разделить A на $L = \{x < y\}$, $E = \{x = y\}$, $R = \{x > y\}$.
5. Если $i \leq |L|$, вызвать $\text{Select}(L, i)$; если $i \leq |L| + |E|$, вернуть y ; иначе вызвать $\text{Select}(R, i - |L| - |E|)$.

7.3 Почему разбиение сбалансировано

Не менее половины медиан не меньше y . В каждой полной пятёрке с такой медианой не менее трёх элементов не меньше y . Поэтому не менее

$$3 \left\lfloor \frac{1}{2} \left\lfloor \frac{n}{5} \right\rfloor \right\rfloor = \frac{3n}{10} - O(1)$$

элементов не меньше y . Аналогично, не менее $3n/10 - O(1)$ элементов не превосходят y . Следовательно, большая рекурсивная часть имеет размер не более $7n/10 + O(1)$.

7.4 Корректность

Доказательство / обоснование. Partition сохраняет все элементы и точно определяет, сколько элементов строго меньше y и сколько равно y . Если ранг попадает в L , искомый элемент имеет тот же ранг внутри L . Если он попадает в блок равных элементов, ответ y . В правой части из ранга вычитается число элементов, гарантированно стоящих раньше. Индукция по размеру массива завершает доказательство.

7.5 Линейное время

Рекурсия имеет вид

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10 + O(1)) + cn.$$

Суммарный размер рекурсивных задач меньше n :

$$\frac{n}{5} + \frac{7n}{10} = \frac{9n}{10}.$$

Проверка подстановкой: для достаточно большого C предположим $T(k) \leq Ck$ при $k < n$. Тогда

$$T(n) \leq C\frac{n}{5} + C\frac{7n}{10} + cn = \left(\frac{9C}{10} + c\right)n \leq Cn$$

при $C \geq 10c$. Значит, $T(n) = O(n)$. Нижняя оценка $\Omega(n)$ очевидна: в общем случае нужно просмотреть все элементы.

7.6 Случайный Quickselect

Можно выбирать опорный случайно и рекурсировать только в одной части. Ожидаемое время $O(n)$, но худшее $O(n^2)$. На практике randomised quickselect обычно проще и быстрее BFPRT, тогда как медиана медиан даёт строгую гарантию.

Каркас устного ответа

Определить порядковую статистику; описать пятёрки и медиану медиан; показать наличие по $3n/10 - O(1)$ элементов с каждой стороны; получить рекурсию $T(n) \leq T(n/5) + T(7n/10) + O(n) = O(n)$; объяснить переход ранга после partition.

Билет 8. Поиск в графе: BFS, DFS, топологическая сортировка и КСС

Ключевая идея

BFS обходит граф слоями и находит кратчайшие пути в невзвешенном графе. DFS строит лес вложенных временных интервалов; времена завершения дают топологический порядок и используются в алгоритме компонент сильной связности.

8.1 Представление графа

При списках смежности все вершины и дуги можно просмотреть за $\Theta(|V| + |E|)$. Для ориентированной дуги (u, v) вершина v находится в списке исходящих соседей u .

8.2 Поиск в ширину

BFS поддерживает очередь. При первом обнаружении вершины v записываются расстояние $d[v] = d[u] + 1$ и предок $p[v] = u$.

Алгоритм: BFS(G, s)

1. Пометить s , положить $d[s] = 0$, добавить s в очередь.
2. Пока очередь не пуста, извлечь u .
3. Для каждой дуги (u, v) : если v не помечена, пометить её, присвоить $d[v] = d[u] + 1$, $p[v] = u$ и добавить в очередь.

Инвариант. В очереди сначала находятся оставшиеся вершины некоторого слоя ℓ , затем вершины слоя $\ell + 1$. Все вершины меньших слоёв уже обработаны.

Доказательство / обоснование. Новая вершина из соседа слоя ℓ имеет путь длины $\ell + 1$. Если бы существовал более короткий путь, её предшественник на этом пути был бы обработан раньше и вершина уже была бы открыта. Поэтому первое присвоенное расстояние минимально.

Сложность. $O(|V| + |E|)$, поскольку каждая вершина входит в очередь один раз, а каждая дуга просматривается один раз.

8.3 Поиск в глубину

DFS идёт по первому ещё не исследованному ребру и возвращается, когда продолжать некуда. Цвета: белый — не открыт, серый — находится в стеке рекурсии, чёрный — полностью обработан. Записываются времена открытия $tin[v]$ и завершения $tout[v]$.

Алгоритм: DFS-Visit(u)

1. Покрасить u в серый и записать $tin[u]$.
2. Для каждой дуги (u, v) , если v белая, положить $p[v] = u$ и вызвать DFS-Visit(v).
3. Покрасить u в чёрный и записать $tout[u]$.

Интервалы $[tin[v], tout[v]]$ для двух вершин либо не пересекаются, либо один вложен в другой. Вложение означает отношение предок-потомок в DFS-лесе.

Классификация дуг: древесные, обратные (в предка, то есть в серую вершину), прямые и перекрёстные. Ориентированный граф ациклический тогда и только тогда, когда DFS не встречает обратных дуг.

8.4 Топологическая сортировка

Для DAG запускаем DFS из всех ещё не посещённых вершин и выписываем вершины в порядке убывания $tout$.

Доказательство / обоснование. Для любой дуги (u, v) в DAG должно быть $tout[u] > tout[v]$. Если v белая при просмотре дуги, она завершится внутри вызова из u . Если v уже чёрная, она завершилась раньше. Серой быть не может, иначе дуга была бы обратной и создавала цикл. Следовательно, u располагается раньше v при убывании времени завершения.

8.5 Компоненты сильной связности

Вершины u, v сильно связны, если существуют пути $u \rightsquigarrow v$ и $v \rightsquigarrow u$. Компоненты сильной связности (КСС) образуют разбиение V . Если каждую КСС стянуть в вершину, получится ациклический граф конденсации.

8.6 Алгоритм Косараджу-Шарира

Алгоритм: SCC

1. Выполнить DFS исходного графа и записать вершины по убыванию времени завершения.
2. Построить транспонированный граф G^T , развернув все дуги.
3. В указанном порядке запускать DFS в G^T из ещё не посещённых вершин.
4. Каждое дерево второго DFS является одной компонентой сильной связности.

Доказательство / обоснование. Рассмотрим граф конденсации. Если в исходном графе есть дуга из компоненты C в D , то при первом DFS максимальное время завершения среди C больше максимального времени среди D либо обе компоненты посещаются из C . Поэтому компоненту с максимальным временем завершения можно считать источником конденсации. В транспонированном графе она становится стоком: DFS из неё не выйдет в другую компоненту, но обойдёт всю свою КСС. После её удаления аргумент повторяется.

Сложность. Два DFS и построение G^T : $O(|V| + |E|)$.

Каркас устного ответа

BFS: очередь, слои, кратчайшие невзвешенные расстояния. DFS: цвета и времена. Топологическая сортировка: убывание *tout* и отсутствие серых рёбер. КСС: два прохода, транспонирование, порядок завершения и доказательство через DAG конденсации.

Билет 9. Кратчайшие пути: Беллман-Форд, Дейкстра и очередь с приоритетами

Ключевая идея

Оба алгоритма основаны на релаксации. Беллман-Форд допускает отрицательные веса и систематически увеличивает допустимое число рёбер пути. Дейкстра навсегда фиксирует ближайшую вершину, что корректно только при неотрицательных весах.

9.1 Релаксация

Для источника s поддерживаются оценки $d[v]$, изначально $d[s] = 0$, $d[v] = +\infty$. Для дуги (u, v) веса $w(u, v)$:

если $d[v] > d[u] + w(u, v)$, то $d[v] \leftarrow d[u] + w(u, v)$, $p[v] \leftarrow u$.

В любой момент конечное $d[v]$ является длиной некоторого найденного пути, поэтому не меньше истинного кратчайшего расстояния $\delta(s, v)$.

9.2 Беллман-Форд

Алгоритм: Bellman-Ford

1. Инициализировать расстояния.
2. $|V| - 1$ раз просмотреть все дуги и релаксировать их.
3. Сделать ещё один проход. Если какая-либо оценка уменьшается, сообщить о достижимом отрицательном цикле.

Доказательство / обоснование. После k полных проходов значение $d[v]$ не превосходит длину кратчайшего пути из s в v , использующего не более k рёбер. Индукция: последний шаг такого пути — дуга (u, v) , а префикс содержит не более $k - 1$ рёбер и уже учтён. Если отрицательных достижимых циклов нет, кратчайший простой путь содержит не более $|V| - 1$ рёбер, значит после $|V| - 1$ проходов все расстояния точны.

Если на дополнительном проходе возможно улучшение, последовательность предков содержит повторную вершину и, следовательно, цикл; уменьшение означает отрицательный суммарный вес. Обратно, достижимый отрицательный цикл позволяет неограниченно уменьшать длину пути и обязательно порождает релаксацию.

Сложность. $O(|V||E|)$ времени, $O(|V|)$ памяти сверх графа.

9.3 Алгоритм Дейкстры

Предполагаются $w(e) \geq 0$. Множество S содержит вершины с окончательно установленными расстояниями; остальные лежат в очереди с приоритетами по ключу $d[v]$.

Алгоритм: Dijkstra

1. Инициализировать $d[s] = 0$, остальные $+\infty$; поместить вершины в priority queue.
2. Извлечь вершину u с минимальным $d[u]$ и добавить её в S .
3. Релаксировать все исходящие дуги (u, v) , выполняя decrease-key.
4. Повторять до пустой очереди.

Доказательство / обоснование. Пусть u — извлечённая вершина с минимальной оценкой и предположим, что существует более короткий путь P из s в u . На P рассмотрим первую вершину $y \notin S$ и её предшественника $x \in S$. После обработки x имеем $d[y] \leq \delta(s, y)$. Из неотрицательности весов $\delta(s, y) \leq \delta(s, u) < d[u]$, что противоречит минимальности ключа u . Поэтому $d[u] = \delta(s, u)$.

Важно. При отрицательном ребре аргумент ломается: более короткий путь к уже извлечённой вершине может прийти через ещё не обработанную вершину.

9.4 Очередь с приоритетами

Нужны операции insert, min, extract-min, decrease-key. Бинарная куча даёт:

$$\text{insert} = O(\log |V|), \quad \text{extract-min} = O(\log |V|), \quad \text{decrease-key} = O(\log |V|).$$

Важно хранить отображение “вершина графа $\leftrightarrow$ позиция в куче”, обновляя его при обменах.

Дейкстра выполняет $|V|$ извлечений и не более $|E|$ уменьшений ключа:

$$O((|V| + |E|) \log |V|).$$

При массиве без кучи: $O(|V|^2 + |E|) = O(|V|^2)$, что выгодно для плотных графов. Фибоначчиева куча теоретически даёт $O(|E| + |V| \log |V|)$ амортизированно.

Каркас устного ответа

Определить релаксацию. Для Беллмана–Форда доказать инвариант «не более k рёбер» и проверку отрицательного цикла. Для Дейкстры сформулировать множество окончательных вершин и доказательство через первое ребро, выходящее из S . Назвать реализации очереди и сложности.

Билет 10. Пути между всеми парами: Варшалл, Флойд-Варшалл и матричное умножение

Ключевая идея

Алгоритмы Варшалла и Флойда–Варшалла добавляют допустимые промежуточные вершины по одной. Булевы степени матрицы смежности кодируют существование путей фиксированной длины.

10.1 Транзитивное замыкание и алгоритм Варшалла

Пусть вершины пронумерованы $1, \dots, n$. Определим

$$R_{ij}^{(k)} = 1$$

тогда и только тогда, когда существует путь из i в j , все внутренние вершины которого принадлежат $\{1, \dots, k\}$. База: $R^{(0)} = I \vee A$, где A — матрица смежности.

Путь, разрешающий вершины до k , либо не использует k , либо проходит через неё:

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \vee (R_{ik}^{(k-1)} \wedge R_{kj}^{(k-1)}).$$

Таблицу можно обновлять на месте тройным циклом k, i, j . Время $\Theta(n^3)$, память $\Theta(n^2)$.

10.2 Флойд-Варшалл

Для взвешенного графа положим $D_{ij}^{(k)}$ равным длине кратчайшего пути из i в j с внутренними вершинами из $\{1, \dots, k\}$. База:

$$D_{ii}^{(0)} = 0, \quad D_{ij}^{(0)} = w(i, j) \text{ при наличии дуги, } +\infty \text{ иначе.}$$

Рекурсия:

$$D_{ij}^{(k)} = \min \left(D_{ij}^{(k-1)}, D_{ik}^{(k-1)} + D_{kj}^{(k-1)} \right).$$

Доказательство / обоснование. Возьмём кратчайший допустимый путь. Если он не содержит k как внутреннюю вершину, работает первый вариант. Если содержит, при отсутствии отрицательных циклов можно считать, что k встречается один раз; путь распадается на оптимальные части $i \rightsquigarrow k$ и $k \rightsquigarrow j$, внутренние вершины которых имеют номера меньше k .

Алгоритм допускает отрицательные рёбра. После выполнения отрицательный цикл достижим из i и ведёт к j , если для некоторой вершины k имеем $D[k, k] < 0$ и существуют конечные пути $i \rightsquigarrow k$, $k \rightsquigarrow j$. Для восстановления пути хранят матрицу следующей вершины или промежуточного разреза.

10.3 Пути и степени матрицы

Для обычного арифметического умножения

$$(A^\ell)_{ij}$$

равно числу ориентированных маршрутов длины ровно ℓ из i в j . Это доказывается индукцией: последний переход приходит из некоторой вершины k , а суммирование по k перебирает все варианты.

В булевом полукольце сложение заменяется на $\vee$, умножение на $\wedge$; тогда $(A^\ell)_{ij} = 1$ означает существование пути длины ровно ℓ . Достижимость кодируется

$$I \vee A \vee A^2 \vee \dots \vee A^{n-1}.$$

Поскольку любой достижимый путь можно сократить до простого, $n - 1$ рёбер достаточно.

Один способ — вычислить булеву степень $(I \vee A)^{n-1}$ быстрым возведением в степень: её единица означает существование маршрута длины не более $n - 1$ с добавлением петель из I . Если умножение булевых матриц размера n занимает $M(n)$, время составляет $O(M(n) \log n)$. При обычном умножении это $O(n^3 \log n)$, что хуже Варшалла, но с быстрым матричным умножением даёт теоретическое ускорение.

10.4 Полукольцевая точка зрения

Варшалл работает над полукольцом $(\{0, 1\}, \vee, \wedge)$, Флойд-Варшалл — над тропическим полукольцом $(\mathbb{R} \cup \{+\infty\}, \min, +)$. Одна и та же схема “либо без k , либо через k ” объясняет сходство алгоритмов.

Каркас устного ответа

Для Варшалла определить $R^{(k)}$ и рекурсию через k . Для Флойда определить $D^{(k)}$ и заменить $\vee, \wedge$ на $\min, +$. Затем объяснить, что элементы A^ℓ считают/обнаруживают пути длины ℓ , а транзитивное замыкание получается из степеней до $n - 1$.

Билет 11. Быстрое умножение матриц: алгоритм Штрассена

Ключевая идея

Обычное блочное умножение требует восьми произведений половинных матриц. Штрассен выражает результат через семь произведений, уменьшая показатель сложности с 3 до $\log_2 7 \approx 2.807$.

11.1 Блочное представление

Для $n = 2^k$ разобьём

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}.$$

Обычное блочное умножение вычисляет четыре блока результата через восемь умножений матриц размера $n/2$.

11.2 Семь произведений Штрассена

Одна стандартная нумерация:

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), \\ M_2 &= (A_{21} + A_{22})B_{11}, \\ M_3 &= A_{11}(B_{12} - B_{22}), \\ M_4 &= A_{22}(B_{21} - B_{11}), \\ M_5 &= (A_{11} + A_{12})B_{22}, \\ M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}). \end{aligned}$$

Тогда

$$\begin{aligned} C_{11} &= M_1 + M_4 - M_5 + M_7, \\ C_{12} &= M_3 + M_5, \\ C_{21} &= M_2 + M_4, \\ C_{22} &= M_1 - M_2 + M_3 + M_6. \end{aligned}$$

Доказательство / обоснование. Корректность проверяется раскрытием скобок. Например, в C_{11} все смешанные члены сокращаются, остаётся $A_{11}B_{11} + A_{12}B_{21}$. Аналогично для остальных блоков. Формулы используют сложение и вычитание, поэтому работают над кольцом, но не непосредственно над произвольным полукольцом.

11.3 Сложность

На каждом уровне выполняются семь рекурсивных умножений и $O(n^2)$ операций сложения блоков:

$$T(n) = 7T(n/2) + O(n^2).$$

Поскольку $n^{\log_2 7}$ растёт быстрее n^2 ,

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807}).$$

Если размер не является степенью двойки, матрицы дополняются нулевыми строками и столбцами.

11.4 Практические замечания

Штрассен выгоден только выше порога: он выполняет больше сложений, сложнее использует кэш и может хуже вести себя численно. На нижних уровнях переходят к классическому алгоритму. Теоретически показатель матричного умножения ω сегодня меньше 2.373, но соответствующие алгоритмы непрактичны.

Каркас устного ответа

Разбить матрицы на четыре блока; отметить восемь обычных произведений; выписать семь M_i и четыре формулы восстановления; сказать, что корректность проверяется раскрытием; решить $T(n) = 7T(n/2) + O(n^2)$.

Билет 12. Булевы матрицы: сведение к числовым и метод четырёх русских

Ключевая идея

Булево произведение можно получить из числового, проверяя ненулевость суммы произведений. Метод четырёх русских группирует столбцы и заранее вычисляет ответы для всех битовых шаблонов, экономя логарифмический множитель.

12.1 Булево произведение

Для $A \in \{0, 1\}^{n \times m}$ и $B \in \{0, 1\}^{m \times p}$:

$$C_{ij} = \bigvee_{k=1}^m (A_{ik} \wedge B_{kj}).$$

Элемент C_{ij} равен единице тогда и только тогда, когда существует индекс k с $A_{ik} = B_{kj} = 1$.

12.2 Через числовое умножение

Рассмотрим те же матрицы над целыми числами. Тогда

$$D_{ij} = \sum_{k=1}^m A_{ik} B_{kj}$$

— число свидетелей k . Следовательно,

$$C_{ij} = 1 \iff D_{ij} > 0.$$

Можно считать по модулю $q > m$, например $q = m + 1$, чтобы никакое положительное значение $1, \dots, m$ не превратилось в ноль. После быстрого числового умножения каждый элемент заменяется индикатором ненулевости. В арифметической модели это даёт $O(n^\omega)$ для квадратных матриц.

Важно. Модуль должен превосходить максимальную возможную сумму. При вычислении по модулю 2 чётное число свидетелей исчезло бы, и ответ стал бы неверным.

12.3 Метод четырёх русских

Пусть квадратные матрицы имеют размер n . Выберем размер блока $t \approx \log_2 n$ и разобьём внутренний индекс k на $q = \lceil n/t \rceil$ блоков.

Для каждого блока строк матрицы B , состоящего из t строк, заранее вычислим побитовое OR всех подмножеств этих строк. Всего шаблонов 2^t . Таблицу можно построить за $O(2^t n)$ времени на блок, например динамически по младшему установленному биту.

Строка A_i на данном блоке задаёт t -битовую маску s . Частичный вклад в строку результата равен заранее вычисленному объединению строк B с номерами, где в маске стоят единицы. Полный результат получается OR частичных вкладов по всем блокам.

12.4 Анализ

Предвычисление для всех q блоков:

$$O\left(\frac{n}{t} 2^t n\right).$$

Основная часть: для каждой из n строк и каждого из n/t блоков объединяется битовый вектор длины n , то есть в простой модели

$$O\left(\frac{n^3}{t}\right).$$

При $t = \lfloor \log_2 n \rfloor$ имеем $2^t \leq n$, и предвычисление $O(n^3/t)$ того же порядка. Итого

$$O\left(\frac{n^3}{\log n}\right).$$

В word-RAM с упаковкой битов оценка может быть улучшена дополнительным фактором размера машинного слова.

12.5 Почему метод работает

Для фиксированной строки i , столбца j и блока K частичный результат равен

$$\bigvee_{k \in K} (A_{ik} \wedge B_{kj}).$$

Маска строки A выбирает ровно те строки блока B , для которых $A_{ik} = 1$; предвычисленный OR даёт эту дизъюнкцию сразу. OR по блокам восстанавливает дизъюнкцию по всем k .

Каркас устного ответа

Определить булево произведение. Для сведения к числовому объяснить, что сумма считает свидетелей, и взять модуль $> n$. Для четырёх русских разделить внутренний индекс на блоки t , предвычислить OR для 2^t масок и получить $O(n^3/t)$, выбрав $t = \Theta(\log n)$.

Билет 13. Представление множеств: вектор, список, двоичное дерево поиска, AVL-дерево**Ключевая идея**

Выбор структуры данных определяет компромисс между поиском и изменением множества. Сбалансированное дерево поиска поддерживает `find`, `insert` и `erase` за $O(\log n)$; AVL-дерево обеспечивает баланс локальными вращениями.

13.1 Абстрактный тип «множество»

Обычно нужны операции:

`find(x)`, `insert(x)`, `erase(x)`,

а для упорядоченного множества также `minimum`, `maximum`, `predecessor`, `successor` и обход в отсортированном порядке.

13.2 Массив и связный список

Неупорядоченный динамический массив: поиск $O(n)$, вставка в конец амортизированно $O(1)$, удаление найденного элемента $O(1)$ заменой последним, если порядок не важен. Вставка с расширением иногда стоит $O(n)$, но суммарно за n вставок — $O(n)$.

Отсортированный массив: поиск $O(\log n)$ бинарным поиском, но вставка и удаление требуют сдвига $O(n)$. Зато компактное хранение хорошо использует кэш.

Связный список: вставка/удаление по известному указателю $O(1)$, но поиск $O(n)$; бинарный поиск невозможен из-за отсутствия произвольного доступа. Память включает указатели.

13.3 Двоичное дерево поиска

Для каждой вершины с ключом x все ключи левого поддерева меньше x , правого — больше. Поиск идёт по одному пути. Вставка заканчивает неуспешный поиск созданием листа.

Удаление:

- лист удаляется непосредственно;
- вершина с одним ребёнком заменяется ребёнком;
- вершина с двумя детьми заменяется следующим по величине элементом — минимумом правого поддерева, после чего удаляется этот минимум.

Все операции занимают $O(h)$, где h — высота. Для случайного порядка вставок ожидается $O(\log n)$, но монотонный порядок создаёт цепь высоты n .

13.4 AVL-дерево

AVL-дерево — BST, в каждой вершине которого высоты левого и правого поддеревьев отличаются не более чем на единицу:

$$\text{bf}(v) = h(v_R) - h(v_L) \in \{-1, 0, 1\}.$$

В вершинах хранят высоту или balance factor.

13.5 Логарифмическая высота

Пусть $N(h)$ — минимальное число вершин в AVL-дереве высоты h . Для минимального дерева высоты поддеревьев равны $h-1$ и $h-2$:

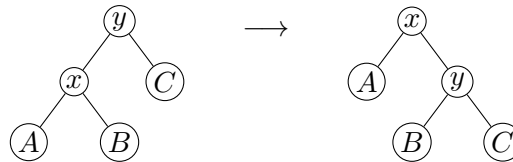
$$N(h) = 1 + N(h-1) + N(h-2).$$

Следовательно, $N(h) = F_{h+3} - 1 = \Omega(\varphi^h)$, где $\varphi = (1 + \sqrt{5})/2$. Поэтому

$$h = O(\log n), \quad h \leq \log_{\varphi}(n+1) - O(1) \approx 1.44 \log_2 n.$$

13.6 Вращения

Правое вращение вокруг y при левом ребёнке x сохраняет порядок ключей:



Действительно, $A < x < B < y < C$ до и после вращения.

После вставки поднимаемся от нового листа к корню, обновляя высоты. Если вершина разбалансирована:

- LL: одно правое вращение;
- RR: одно левое вращение;
- LR: левое вращение ребёнка, затем правое вершины;
- RL: симметрично.

После вставки достаточно исправить первую разбалансированную вершину. После удаления разбалансировка может распространяться выше, поэтому возможны $O(\log n)$ вращений.

Доказательство / обоснование. Вращение сохраняет inorder-порядок, значит свойство BST. Выбор одинарного или двойного вращения по знакам balance factor перераспределяет три упорядоченных корня и четыре поддерева так, чтобы высоты новых детей различались не более чем на единицу. Выше по пути изменяется только высота исправленного поддерева.

Сложность. find, insert, erase, predecessor, successor — $O(\log n)$; память $O(n)$.

Структура	find	insert	erase
Неупорядоченный массив	$O(n)$	аморт. $O(1)$	$O(n)$ на поиск
Отсортированный массив	$O(\log n)$	$O(n)$	$O(n)$
Список	$O(n)$	$O(1)$ по месту	$O(1)$ по месту
Обычный BST	$O(h)$	$O(h)$	$O(h)$
АВЛ-дерево	$O(\log n)$	$O(\log n)$	$O(\log n)$

Каркас устного ответа

Сравнить массив, список и BST. Для BST описать инвариант и три случая удаления. Определить АВЛ-баланс, доказать логарифмическую высоту через числа Фибоначчи, показать одно и двойное вращение и назвать сложности.

Билет 14. В-деревья и понятие о красно-чёрных деревьях

Ключевая идея

В-дерево минимизирует число обращений к внешней памяти, помещая много ключей в один узел-блок. Красно-чёрное дерево — двоичное сбалансированное дерево с более слабым балансом, чем АВЛ, но дешёвыми обновлениями.

14.1 Модель внешней памяти

Диск или SSD читает данные блоками. Если один узел структуры занимает целый блок, основная стоимость — число посещённых узлов, а не число сравнений внутри узла. Поэтому выгодно увеличить ветвление дерева.

14.2 Определение В-дерева

Зафиксируем минимальную степень $t \geq 2$. В стандартной формулировке:

- каждый узел содержит от $t - 1$ до $2t - 1$ ключей, кроме корня, которому разрешено меньше;
- ключи в узле упорядочены;
- внутренний узел с m ключами имеет $m + 1$ детей, интервалы ключей которых разделены ключами узла;
- все листья находятся на одной глубине.

В лекционной формулировке через степень узла: не-корневой узел имеет от t до $2t$ детей и на один ключ меньше.

14.3 Высота

Наименьшее число ключей при высоте h получается при минимальной заполненности. Число узлов растёт как $\Omega(t^h)$, поэтому

$$h = O(\log_t n).$$

Если t выбирается так, чтобы узел занимал один дисковый блок, поиск требует $O(\log_t n)$ чтений блоков. Внутри узла нужный интервал находится бинарным поиском за $O(\log t)$ сравнений.

14.4 Поиск

В текущем узле ищем первую позицию i с $x \leq k_i$. Если $x = k_i$, успех; иначе спускаемся в соответствующего ребёнка. Корректность следует из интервалов, задаваемых отсортированными ключами узла.

14.5 Вставка сверху вниз

Проблема: в полный узел с $2t - 1$ ключами нельзя добавить ключ. Перед спуском в полного ребёнка он расщепляется:

- медианный ключ поднимается в родителя;
- $t - 1$ меньших ключей образуют левый узел;
- $t - 1$ больших — правый.

Если корень полон, создаётся новый корень и старый корень расщепляется, высота увеличивается на один. После предварительного расщепления алгоритм никогда не спускается в полный узел, поэтому в листе есть место.

Сложность. На каждом уровне постоянное число блочных операций; всего $O(\log_t n)$ I/O. Перемещения ключей внутри блока обычно считаются $O(t)$ в RAM, но одной-двумя операциями ввода-вывода.

14.6 Удаление

Удаление также организуется сверху вниз: перед спуском гарантируется, что ребёнок содержит хотя бы t ключей, а не минимальные $t - 1$. Если он минимален:

- взять ключ у соседнего брата через родителя, если брат имеет запас;
- иначе слить ребёнка с братом и разделяющим ключом родителя.

Если удаляемый ключ находится во внутреннем узле, его заменяют предшественником или преемником из ребёнка с запасом; иначе два соседних ребёнка сливаются. Корень без ключей заменяется единственным ребёнком, и высота уменьшается.

14.7 Красно-чёрное дерево

Это BST, каждому узлу назначен цвет. Стандартные инварианты:

1. корень чёрный;
2. NIL-листья считаются чёрными;

3. у красной вершины дети чёрные;
4. любой путь от вершины до NIL-листа содержит одинаковое число чёрных вершин.

Если чёрная высота корня равна b , дерево содержит не менее $2^b - 1$ внутренних вершин, а между двумя чёрными вершинами не более одной красной. Поэтому

$$h \leq 2b \leq 2\log_2(n+1).$$

Вставка и удаление исправляют цвета и выполняют локальные вращения за $O(\log n)$. Красно-чёрное дерево допускает большую высоту, чем АВЛ, но обычно требует меньше вращений; его варианты лежат в основе стандартных ordered map/set.

Каркас устного ответа

Объяснить мотивацию блоковой памяти; определить диапазон заполненности и одинаковую глубину листьев; вывести $O(\log_t n)$; описать split при вставке и заимствование/merge при удалении. Для красно-чёрного дерева перечислить инварианты и вывести высоту не более $2\log_2(n+1)$.

Билет 15. Полиномиальное хэширование и строковые задачи

Ключевая идея

Строка интерпретируется как значение полинома в случайной точке. Префиксные хэши позволяют сравнивать подстроки за $O(1)$ с малой вероятностью коллизии, что ускоряет поиск образца и задачи на подстроки.

15.1 Полиномиальный хэш

Пусть символам сопоставлены числа, выбран модуль p и основание q . Для строки $w = w_0 \cdots w_{n-1}$ можно определить

$$H(w) = \sum_{i=0}^{n-1} w_i q^i \pmod{p}$$

или эквивалентную схему Горнера. Если строки равны, хэши равны; обратное может нарушаться из-за коллизии.

При случайном $q \in \mathbb{F}_p$ для двух различных строк длины не более n разность хэшей — ненулевой многочлен степени $< n$. Он имеет не более $n - 1$ корней, поэтому

$$\Pr[H(u) = H(v)] \leq \frac{n-1}{p}.$$

На практике используют большой простой модуль, иногда два независимых модуля или 64-битное переполнение.

15.2 Префиксные хэши

При схеме

$$P[i+1] = P[i]q + w_i \pmod{p}$$

хэш подстроки $w[l..r]$ равен

$$H(l, r) = P[r] - P[l]q^{r-l} \pmod{p}.$$

Степени q^k предвычисляются. После $O(n)$ подготовки любой хэш вычисляется за $O(1)$.

15.3 Алгоритм Рабина-Карпа

Для текста T длины n и образца X длины m вычисляется $H(X)$ и хэши всех окон $T[i..i+m)$. Совпадающие хэши считаются кандидатами; при детерминированной версии кандидат проверяется посимвольно.

Сложность. С префиксными хэшами — $O(n+m)$ ожидаемого/вероятностного времени. При обязательной проверке худший случай $O(nm)$ из-за большого числа коллизий, но результат всегда корректен. Без проверки время линейное, но возможна ошибка.

15.4 Наибольшая общая подстрока

Для строк u, v проверим, существует ли общая подстрока длины L :

1. занести хэши всех подстрок u длины L в хэш-таблицу;
2. просмотреть подстроки v длины L и искать совпадающий хэш;
3. при необходимости проверить символы.

Свойство монотонно: если есть общая подстрока длины L , то есть и любой меньшей длины. Поэтому бинарный поиск по L даёт $O((|u|+|v|) \log \min(|u|, |v|))$ ожидаемого времени и линейную память. При сортировке массивов хэшей вместо таблицы появляется дополнительный логарифм.

15.5 Самый длинный палиндром

Построим хэши строки w и обратной строки w^R . Подстрока $w[l..r)$ является палиндромом тогда и только тогда, когда она равна соответствующей подстроке обратной строки:

$$w[l..r) = w^R[n-r..n-l).$$

Это проверяется за $O(1)$ по хэшам.

Для каждого центра отдельно рассматриваются нечётные и чётные палиндромы. Максимальный радиус находится бинарным поиском, поскольку истинность свойства монотонна по радиусу. Всего $O(n \log n)$ времени и $O(n)$ памяти. Существуют линейные алгоритмы, например Манакера, но в данном билете важна реализация через полиномиальные хэши.

Важно. При модульном вычитании нужно нормализовать остаток. Сравнивать хэши подстрок разных длин без согласованной схемы нельзя. Вероятностную природу метода и способ устранения ошибки проверкой следует проговорить.

Каркас устного ответа

Определить полиномиальный хэш и оценку коллизии через число корней многочлена. Вывести формулу префиксного хэша. Рабин-Карп: сравнение окон. Для общей подстроки — проверка фиксированной длины и бинарный поиск; для палиндрома — сравнение с обратной строкой и поиск радиуса.

Билет 16. Алгоритм Кнута-Морриса-Пратта и конечный автомат**Ключевая идея**

КМР после несовпадения не возвращается по тексту назад: он заменяет совпавший префикс образца на его самый длинный собственный префикс, одновременно являющийся суффиксом. Это состояние естественно интерпретируется как состояние конечного автомата.

16.1 Префиксная функция

Для образца $x = x_0 \dots x_{m-1}$ определим $\pi[i]$ как длину наибольшего собственного префикса строки $x[0..i]$, совпадающего с её суффиксом. Например, цепочка

$$i, \pi[i] - 1, \pi[\pi[i] - 1] - 1, \dots$$

перечисляет длины возможных границ.

Вычисление π :

1. для $i = 1, \dots, m - 1$ положить $j = \pi[i - 1]$;
2. пока $j > 0$ и $x_i \neq x_j$, заменить $j = \pi[j - 1]$;
3. если $x_i = x_j$, увеличить j ;
4. положить $\pi[i] = j$.

Амортизированный анализ: j увеличивается не более чем на единицу на итерацию, а каждое уменьшение компенсируется предыдущими увеличениями. Поэтому вся таблица строится за $O(m)$.

16.2 Поиск

Поддерживается j — длина наибольшего префикса образца, являющегося суффиксом прочитанной части текста. Для символа s выполняются те же откаты по π , затем при совпадении j увеличивается. Если $j = m$, найдено вхождение, после чего для поиска перекрывающихся вхождений берётся $j = \pi[m - 1]$.

Инвариант. После обработки $T[0..i]$ значение j равно максимальной длине k , для которой $x[0..k) = T[i - k + 1..i + 1)$.

Доказательство / обоснование. Если следующий символ совпадает, максимальный суффикс продлевается. При несовпадении любой меньший подходящий префикс обязан быть границей уже совпавшего префикса, поэтому перебирается цепочкой π . Первый продлеваемый кандидат максимален; если кандидатов нет, остаётся состояние 0.

Сложность. Предобработка $O(m)$, поиск $O(n)$, память $O(m)$.

16.3 Автоматная реализация

Состояния $0, 1, \dots, m$ обозначают длину совпавшего префикса. Переход

$$\delta(q, a) = \max\{k \leq m : x[0..k) \text{ является суффиксом } x[0..q)a\}.$$

Состояние m — принимающее: образец заканчивается в текущей позиции. Текст читается одним проходом по переходам автомата.

Таблицу δ можно построить за $O(m|\Sigma|)$ с помощью префиксных ссылок:

$$\delta(q, a) = \begin{cases} q + 1, & q < m \text{ и } x_q = a, \\ \delta(\pi[q - 1], a), & q > 0 \text{ и нет прямого совпадения,} \\ 0, & q = 0 \text{ и } x_0 \neq a. \end{cases}$$

После построения поиск занимает ровно n переходов. Обычный КМР не хранит всю таблицу: он вычисляет нужные переходы на лету, экономя фактор $|\Sigma|$ в памяти и предобработке.

Каркас устного ответа

Определить π как длину максимальной границы; описать линейное построение и амортизацию; сформулировать инвариант КМР. Затем интерпретировать j как состояние автомата, дать определение $\delta(q, a)$ и сравнить полную таблицу с вычислением переходов по failure links.

Билет 17. Префиксное дерево и алгоритм Ахо-Корасик

Ключевая идея

Trie объединяет общие префиксы множества образцов. Ахо-Корасик добавляет суффиксные ссылки, превращая trie в автомат, который за один проход находит все вхождения всех образцов.

17.1 Префиксное дерево

Каждая вершина trie соответствует префиксу некоторой хранимой строки, корень — пустой строке. Дуга с меткой a добавляет символ a . Конечные вершины отмечают завершение слов.

Вставка и поиск строки x требуют $O(|x|)$ времени. Число вершин не превосходит суммарной длины слов плюс один. Рёбра можно хранить массивом по алфавиту, словарём или отсортированным списком — это влияет на константы и фактор поиска перехода.

17.2 Суффиксная ссылка

Для вершины v , соответствующей строке u , failure link $link[v]$ указывает на вершину самого длинного собственного суффикса u , который является префиксом некоторого образца. Это обобщение префиксной функции КМР.

Кроме того, в вершине нужно знать все образцы, заканчивающиеся в ней или в вершинах цепочки суффиксных ссылок. Это реализуют:

- списками output, наследуемыми по ссылке;
- отдельной выходной ссылкой на ближайшую терминальную вершину;
- подсчётом посещений с последующим распространением по дереву ссылок.

17.3 Построение автомата

Ссылки строятся BFS по глубине trie. Для детей корня ссылка ведёт в корень. Для ребра $v \xrightarrow{a} u$:

1. начать из $link[v]$;
2. переходить по failure links, пока не найдётся ребро по a ;
3. ссылка u ведёт в найденный переход; если его нет даже из корня, в корень.

Можно сразу заполнить полную функцию переходов: отсутствующий переход из v по a равен переходу из $link[v]$ по a . Тогда построение занимает $O(S|\Sigma|)$, где S — число вершин; при словарях и вычислении только существующих переходов — около $O(S \log |\Sigma|)$ или линейно для фиксированного алфавита.

Алгоритм: Поиск Ахо-Корасика

1. Начать в корне.
2. Для каждого символа текста перейти по автоматному переходу; если полной таблицы нет, откатываться по failure links.
3. Вывести образцы из output текущей вершины.

17.4 Корректность

Инвариант. После чтения префикса текста текущая вершина соответствует самому длинному суффиксу прочитанного текста, являющемуся префиксом какого-либо образца.

Доказательство / обоснование. Прямой переход продлевает текущий префикс. Если его нет, любой допустимый суффикс обязан быть суффиксом текущей строки и одновременно префиксом образца; failure link выбирает самый длинный такой кандидат. Повторение приводит к максимальному продлеваемому кандидату. Все образцы, заканчивающиеся в текущей позиции, являются терминальными суффиксами текущей вершины и перечисляются по output-ссылкам.

Сложность. Для фиксированного алфавита построение $O(S|\Sigma|)$ с полной таблицей, поиск $O(n + z)$, где z — число выведенных вхождений. Без полной таблицы амортизированное число откатов также линейно. Память $O(S|\Sigma|)$ или $O(S)$ рёбер плюс ссылки.

Каркас устного ответа

Определить trie и стоимость операций. Ввести failure link как максимальный суффикс-префикс. Построить ссылки BFS, объяснить output. Сформулировать инвариант максимального суффикса и получить $O(n + z)$ поиска.

Билет 18. Сжатие Хаффмана и арифметическое кодирование

Ключевая идея

Хаффман строит оптимальный посимвольный префиксный код с целыми длинами кодовых слов. Арифметическое кодирование кодирует всю строку одним вложенным интервалом и приближается к информационной длине $-\log P(w)$.

18.1 Префиксные коды

Код $c : \Sigma \rightarrow \{0, 1\}^*$ префиксный, если ни одно кодовое слово не является префиксом другого. Тогда декодирование однозначно слева направо. Код представляется двоичным деревом: символы в листьях, код — путь от корня.

Для частот f_a длина закодированного сообщения:

$$L = \sum_{a \in \Sigma} f_a \ell_a,$$

где ℓ_a — глубина листа.

18.2 Алгоритм Хаффмана

Алгоритм: Huffman

1. Создать по листу веса f_a для каждого символа.
2. Поместить деревья в min-priority queue.
3. Пока деревьев больше одного, извлечь два минимальных веса x, y , соединить их новым корнем веса $x + y$ и вернуть в очередь.
4. Пометить два ребра каждого объединения битами 0 и 1.

Сложность. При $\sigma = |\Sigma|$ двоичная куча даёт $O(\sigma \log \sigma)$; кодирование строки после построения — $O(n)$.

18.3 Оптимальность Хаффмана

Доказательство / обоснование. В оптимальном полном кодовом дереве два наименее частых символа можно поместить в два листа максимальной глубины с общим родителем, не увеличив стоимость: обмен кодами переносит редкие символы на не меньшую глубину.

Склеим эти символы a, b в метасимвол веса $f_a + f_b$. Удаление их последнего бита превращает код в префиксный код меньшего алфавита, а стоимость уменьшается ровно на $f_a + f_b$. По индукции оптимальный код меньшего алфавита строится Хаффманом; обратное расщепление метасимвола даёт оптимальный исходный код. Именно этот жадный шаг выполняет алгоритм.

Хаффман оптимален среди префиксных кодов, кодирующих каждый символ отдельным целым числом битов, но не обязательно среди всех методов сжатия.

18.4 Арифметическое кодирование

Пусть символы имеют вероятности $P(a)$, сумма равна единице. Начинаем с интервала $[l, r) = [0, 1)$. Для очередного символа a делим текущий интервал на части длины $(r - l)P(b)$ в порядке алфавита и выбираем часть a :

$$l' = l + (r - l) \sum_{b < a} P(b),$$

$$r' = l + (r - l) \sum_{b \leq a} P(b).$$

После всей строки выбирается любое двоичное число внутри итогового интервала, а декодер, зная модель вероятностей и длину сообщения/стоп-символ, повторяет разбиения.

Длина интервала после строки $w = a_1 \cdots a_n$ равна

$$\prod_{i=1}^n P(a_i) = P(w).$$

Интервал длины $P(w)$ можно идентифицировать примерно $-\log_2 P(w) + O(1)$ битами. При независимой модели средняя длина близка к энтропии.

18.5 Практическая реализация

Хранить рациональные числа не нужно. Используется целочисленный интервал $[L, H]$ фиксированной разрядности. Когда старшие биты L и H совпадают, этот бит выводится и интервал масштабируется. Случай, когда интервал зажат около $1/2$ и старшие биты пока различны, обрабатывается отложенными противоположными битами (E3-renormalization). Вероятности часто адаптивно обновляются.

	Хаффман	Арифметическое кодирование
Единица кодирования	символ	вся последовательность/контекст
Длина вклада	целое число битов	фактически дробное среднее
Структура	дерево	вложенные интервалы
Преимущество	простота и скорость	близость к энтропии

Каркас устного ответа

Определить префиксный код и дерево. Описать объединение двух минимальных весов и доказать жадный выбор склеиванием редких символов. Для арифметического кодирования дать формулы нового интервала, объяснить декодирование и связь длины с $-\log P(w)$, упомянуть целочисленную нормализацию.

Билет 19. Суффиксное дерево, применения и алгоритм Укконена

Ключевая идея

Суффиксное дерево — сжатое trie всех суффиксов строки. Оно содержит линейное число вершин и после линейного построения позволяет решать многие задачи о подстроках пропорционально длине запроса. Укконен строит дерево слева направо, используя неявные деревья, суффиксные ссылки и амортизацию.

19.1 Суффиксный trie и сжатие путей

Для строки $w = w_1 \cdots w_n$ обычно добавляют уникальный терминатор $\$$, отсутствующий в алфавите. Суффиксный trie содержит строки

$$w_i w_{i+1} \cdots w_n \$, \quad i = 1, \dots, n + 1.$$

У него может быть $\Theta(n^2)$ вершин. Суффиксное дерево получается сжатием каждого максимального пути из вершин степени один в одно ребро, помеченное целой подстрокой.

Метки рёбер не хранятся как отдельные строки: достаточно пары индексов (l, r) исходной строки. Поэтому объём памяти остаётся линейным.

19.2 Линейный размер

После добавления уникального терминатора каждый суффикс заканчивается отдельным листом: листьев $n + 1$. Каждая внутренняя вершина имеет не менее двух детей. В корневом дереве с L листьями и внутренней степенью не менее двух число внутренних вершин не превосходит $L - 1$. Поэтому всего $O(n)$ вершин и рёбер.

19.3 Основные применения

- **Поиск образца x :** идти по рёбрам, сравнивая символы меток. Время $O(|x|)$ для фиксированного алфавита; листья под найденной точкой задают все позиции вхождения.
- **Число вхождений:** хранить число листьев в поддереве.
- **Самая длинная повторяющаяся подстрока:** самая глубокая внутренняя вершина.
- **Наибольшая общая подстрока:** построить обобщённое дерево для $u\#v\$$ и найти самую глубокую вершину, в поддереве которой есть листья обоих цветов.
- **Лексикографический список суффиксов:** DFS по детям в порядке символов даёт суффиксный массив.

19.4 Почему наивное построение квадратично

Если последовательно вставлять все суффиксы с корня, суммарная длина суффиксов равна $\Theta(n^2)$. Укконен переиспользует информацию между соседними суффиксами и между соседними префиксами строки.

19.5 Неявное суффиксное дерево

На фазе i алгоритм представляет все суффиксы префикса $w[1..i]$, но без обязательного терминатора. Конец суффикса может лежать внутри ребра; поэтому дерево называется неявным. При добавлении символа w_{i+1} нужно продолжить каждый текущий суффикс.

Три правила расширения:

1. если путь суффикса заканчивается листом, его ребро автоматически удлиняется общим текущим концом;
2. если из конечной точки нет перехода по новому символу, создаётся лист; если точка внутри ребра, ребро предварительно расщепляется;
3. если переход по новому символу уже существует, текущий и все последующие суффиксы уже представлены; фаза прекращается (show-stopper rule).

19.6 Активная точка и суффиксные ссылки

Алгоритм хранит активную точку (v, e, ℓ) : вершину v , первое ребро e и длину пройденной части ребра. Это компактное представление конца очередного ещё не обработанного суффикса.

Суффиксная ссылка внутренней вершины, соответствующей строке $a\alpha$, ведёт в вершину, соответствующую α . После обработки одного суффикса следующий получается удалением первого символа; ссылка позволяет перейти почти сразу, не возвращаясь в корень.

Skip/count: если известна длина пути, по ребру можно перескочить целиком, сравнивая длину, а не каждый символ. Каждый такой переход проходит хотя бы одно полное ребро.

Алгоритм: Схема фазы Укконена

1. Добавить новый символ, увеличить общий правый конец всех листовых рёбер.
2. Пока остаются необработанные суффиксы, проверить переход из активной точки по новому символу.
3. Если перехода нет, создать лист; при необходимости расщепить ребро и связать предыдущую созданную внутреннюю вершину суффиксной ссылкой.
4. Если переход есть, увеличить активную длину и завершить фазу по правилу 3.
5. Перейти к следующему суффиксу через суффиксную ссылку и канонизировать активную точку skip/count-переходами.

19.7 Корректность

Доказательство / обоснование. Инвариант фазы: неявное дерево представляет все суффиксы текущего префикса. При добавлении символа каждый старый суффикс должен быть продолжен этим символом. Правила 1 и 2 явно создают требуемые продолжения. Если с активной точки уже есть переход по новому символу, более короткие суффиксы также уже имеют такое продолжение благодаря

суффиксной структуре, поэтому правило 3 безопасно прекращает фазу. Расщепление рёбер не меняет множество читаемых путей.

19.8 Линейная амортизированная сложность

На каждой фазе создаётся не более одного нового листа на расширение, а всего листьев $O(n)$; внутренних вершин также $O(n)$. Число переходов по суффиксным ссылкам линейно по числу расширений. При skip/count активная точка продвигается по рёбрам, и суммарный объём продвижений и откатов амортизируется линейно. Для фиксированного алфавита переход из вершины находится за $O(1)$, поэтому построение занимает $O(n)$ времени и памяти. Для словаря переходов появляется фактор стоимости операции словаря.

Важно. Линейность относится к реализации с индексными метками рёбер, общим концом листьев, суффиксными ссылками и skip/count. Простая вставка всех суффиксов остаётся квадратичной.

Каркас устного ответа

Определить сжатое trie суффиксов с терминатором и доказать линейный размер. Перечислить применения. Для Укконена объяснить фазы, неявное дерево, три правила расширения, активную точку, суффиксные ссылки, skip/count и амортизированное $O(n)$.

Билет 20. Сжатие повторяющихся подстрок: LZ77 и LZ78

Ключевая идея

LZ77 заменяет повтор подстроки ссылкой в уже прочитанный текст. LZ78 строит словарь фраз и кодирует новую фразу парой «номер известного префикса, следующий символ». Оба метода адаптивны и не требуют заранее передавать словарь.

20.1 LZ77

Текст читается слева направо. В текущей позиции ищется наиболее длинный префикс непрочитанного остатка, встречавшийся ранее. Он кодируется тройкой

$$(d, \ell, a),$$

где d — расстояние назад до начала совпадения, ℓ — длина копируемого фрагмента, a — следующий новый символ. Разрешение $\ell > d$ позволяет копировать уже порожденные символы и кодировать периодические серии.

Алгоритм: Жадный LZ77

1. В текущей позиции найти самое длинное совпадение с ранее прочитанной частью или со скользящим окном.
2. Вывести ссылку (d, ℓ) и следующий литерал a ; если совпадения нет, вывести литерал.
3. Сдвинуть текущую позицию на $\ell + 1$.

Декодер уже знает предыдущий текст, поэтому копирует ℓ символов с расстояния d и дописывает a . При перекрытии копирование выполняется последовательно.

20.2 Жадность

В модели, где каждая фраза-ссылка имеет одинаковую стоимость, выбор самого длинного допустимого совпадения минимизирует число фраз: если оптимальное кодирование начинается с более короткой фразы, длинная жадная фраза покрывает её и, возможно, несколько следующих фраз; после неё остаётся суффикс не длиннее, чем после соответствующего числа оптимальных фраз. Формальное доказательство строится обменным аргументом/индукцией по оставшейся длине.

20.3 Скользящее окно

Практически поиск ограничивают последними W символами. Это:

- ограничивает число битов для d ;
- ограничивает память;
- позволяет забывать старые данные;
- иногда ухудшает степень сжатия.

Для поиска совпадений используют хэш-цепочки, суффиксные структуры, деревья или специализированные эвристики. Форматы DEFLATE сочетают вариант LZ77 с кодированием Хаффмана.

20.4 LZ78

Словарь изначально содержит пустую строку $T_0 = \varepsilon$. На каждом шаге находится самая длинная строка T_j из словаря, совпадающая с началом непрочитанного текста. Пусть следующий символ — a . Выводится пара (j, a) , а в словарь добавляется

$$T_{\text{extnew}} = T_j a.$$

После этого прочитанная позиция сдвигается на $|T_j| + 1$.

Доказательство / обоснование. Декодер поддерживает тот же словарь. Получив (j, a) , он выводит уже известную строку T_j и символ a , затем добавляет $T_j a$ под следующим номером. Поэтому словари кодера и декодера совпадают по индукции после каждой фразы.

Словарь удобно хранить trie: поиск самой длинной фразы и добавление нового листа выполняются за время, пропорциональное длине прочитанной фразы. Для декодирования достаточно хранить для каждого номера указатель на предка и последний символ; вывод фразы требует пройти цепочку и развернуть её.

20.5 Сравнение

	LZ77	LZ78
Ссылка	позиция/расстояние в тексте	номер фразы в словаре
Словарь	неявный: прошлый текст	явный, растущий trie
Код	(d, ℓ, a) или литерал	(j, a)
Ограничение памяти	скользящее окно	ограничение/сброс словаря
Варианты	DEFLATE, gzip	LZW

Каркас устного ответа

Для LZ77 определить ссылку (d, ℓ, a) , жадное длиннейшее совпадение, перекрытие и скользящее окно. Для LZ78 определить словарь $T_0 = \varepsilon$, код (j, a) и добавление $T_j a$; доказать декодируемость синхронным построением словаря.

Билет 21. Преобразование Берроуза-Вилера и его реализация

Ключевая идея

BWT переставляет символы так, что одинаковые контексты группируют одинаковые предшествующие символы. Преобразование обратимо: стабильное соответствие между последним и первым столбцами задаёт LF-перестановку.

21.1 Определение

Для строки $w = a_1 \cdots a_n$ рассматриваются все n циклических сдвигов. Их сортируют лексикографически в таблицу T . Пусть $L = b_1 \cdots b_n$ — последний столбец, а t — номер строки, равной исходной w . Тогда

$$\text{BWT}(w) = (L, t).$$

Часто вместо номера строки добавляют уникальный минимальный терминатор $\$$; тогда строка с терминатором определяется однозначно, и отдельный индекс не нужен.

21.2 Почему преобразование помогает сжатию

Если подстрока au часто встречается, то многие циклические сдвиги начинаются с одинакового контекста u и имеют предшествующий символ a . После сортировки эти сдвиги стоят рядом, поэтому в L возникают серии одинаковых символов. BWT само по себе не уменьшает длину, но подготавливает данные для move-to-front, RLE, Хаффмана или арифметического кодирования.

21.3 Обратимость через таблицу

Медленный способ: начиная с пустых строк, n раз приписывать слева столбец L и сортировать строки. После n шагов восстанавливается таблица циклических сдвигов. Это доказывает обратимость, но требует $\Theta(n^2 \log n)$ или больше.

21.4 LF-перестановка

Первый столбец F — отсортированная строка L . При стабильной сортировке k -е вхождение символа c в L соответствует k -му вхождению c в F . Пусть

$$C[c] = \#\{j : L_j < c\},$$

а

$$\text{Осс}(c, i) = \#\{j < i : L_j = c\}.$$

При нумерации с нуля LF-переход:

$$LF(i) = C[L_i] + \text{Осс}(L_i, i).$$

При нумерации с единицы добавляется единица. Он переводит строку таблицы, соответствующую сдвигу uc , в строку сдвига cu , то есть сдвигает начало на один символ назад.

Алгоритм: Inverse-BWT(L, t)

1. За один проход посчитать частоты символов и ранги каждого вхождения в L .
2. По частотам вычислить $C[c]$ и массив LF .
3. Положить $i = t$. Для позиций $k = n, n-1, \dots, 1$: записать $w_k = L_i$, затем $i = LF(i)$.
4. Вернуть восстановленную строку.

Доказательство / обоснование. Последний символ строки в строке таблицы i равен L_i . LF переводит к циклическому сдвигу, у которого этот символ перенесён в начало, то есть к строке, соответствующей предыдущей позиции исходного цикла. Повторяя переход, мы читаем символы исходной строки с конца к началу. Стабильное сопоставление одинаковых символов гарантирует выбор правильного вхождения.

Сложность. Обратное преобразование — $O(n + |\Sigma|)$ времени и $O(n + |\Sigma|)$ памяти.

21.5 Быстрое прямое преобразование

Наивная сортировка всех сдвигов требует квадратичной памяти. BWT можно вычислить через суффиксный массив строки ww или суффиксное дерево $ww\$$, рассматривая только суффиксы, задающие n циклических сдвигов. С линейным алгоритмом построения суффиксного дерева получается $O(n)$ для фиксированного алфавита; с обычным суффиксным массивом — $O(n \log n)$ или $O(n)$ в зависимости от алгоритма.

Каркас устного ответа

Определить таблицу циклических сдвигов, последний столбец и индекс строки. Объяснить появление серий. Для обращения ввести первый столбец, одинаковые порядковые номера вхождений, формулу $LF(i) = C[c] + Occ(c, i)$ и восстановление справа налево за $O(n)$.

Билет 22. Геометрические алгоритмы: ближайшая пара, выпуклая оболочка, диаметр

Ключевая идея

Ближайшая пара решается divide-and-conquer с линейным объединением; выпуклая оболочка — просмотром Грэма с поддержанием левых поворотов; наиболее удалённая пара лежит на оболочке и находится вращающимся штангенциркулем.

22.1 Ориентация тройки точек

Для точек a, b, c определим псевдоскалярное произведение

$$\text{cross}(b - a, c - a) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Знак определяет левый поворот, правый поворот или коллинеарность. Это базовая операция построения оболочки.

22.2 Ближайшая пара точек

Наивный перебор — $O(n^2)$. Эффективный алгоритм заранее сортирует точки по x и по y .

Алгоритм: Closest-Pair

1. Разделить множество вертикальной прямой на две равные по числу точек части, сохраняя списки, отсортированные по x и y .
2. Рекурсивно найти расстояния δ_L, δ_R ; положить $\delta = \min(\delta_L, \delta_R)$.
3. Взять полосу ширины 2δ около разделяющей прямой; её точки уже упорядочены по y .
4. Для каждой точки сравнить её только с последующими точками, чья y -координата отличается меньше δ ; достаточно не более 7 следующих точек.

22.3 Почему достаточно константы соседей

Разобьём полосу на квадраты со стороной $\delta/2$. В каждом таком квадрате может быть не более одной точки из каждой половины, иначе две точки одной половины были бы ближе δ . В прямоугольнике ширины 2δ и высоты δ помещается константное число таких ячеек; стандартный подсчёт даёт не более 7 кандидатов после текущей точки в порядке y .

Разделение списков и проверка полосы выполняются за $O(n)$, поэтому

$$T(n) = 2T(n/2) + O(n) = O(n \log n).$$

Первоначальные сортировки также $O(n \log n)$.

22.4 Выпуклая оболочка и просмотр Грэма

Выпуклая оболочка — минимальное выпуклое множество, содержащее все точки; её граница — выпуклый многоугольник.

Алгоритм: Graham-Scan

1. Выбрать самую нижнюю, а при равенстве самую левую точку p_0 .
2. Отсортировать остальные точки по полярному углу вокруг p_0 ; коллинеарные обработать так, чтобы оставить крайнюю.
3. Поддерживать стек вершин предполагаемой оболочки.
4. Для очередной точки p пока последние две точки стека вместе с p образуют не левый поворот, удалить верхушку. Затем добавить p .

Инвариант. Стек задаёт выпуклую цепь от p_0 до последней обработанной крайней точки; удалённая точка лежит внутри или на границе треугольника и не может быть вершиной окончательной оболочки.

Каждая точка добавляется один раз и удаляется не более одного раза, поэтому просмотр линейный; сортировка доминирует: $O(n \log n)$.

22.5 Наиболее удалённая пара

Диаметр множества — максимальное расстояние между двумя точками. Оба конца диаметра являются вершинами выпуклой оболочки: если один конец лежит внутри выпуклого многоугольника, линейная функция расстояния в фиксированном направлении достигает экстремума на границе, и можно найти не более близкую вершину.

После построения оболочки из h вершин применяют метод вращающегося штангенциркуля. Для каждого ребра оболочки рассматривается противолежащая вершина, максимизирующая расстояние/площадь параллелограмма. При переходе к следующему ребру указатель противолежащей вершины движется только вперёд по циклу, пока площадь увеличивается. Все антиподальные пары перебираются за $O(h)$.

Одна типичная проверка для ребра (p_i, p_{i+1}) :

$$|\text{cross}(p_{i+1} - p_i, p_{j+1} - p_i)| > |\text{cross}(p_{i+1} - p_i, p_j - p_i)|,$$

тогда увеличить j . Для каждой встреченной пары обновить максимум квадратов расстояний, избегая квадратного корня.

Сложность. Оболочка $O(n \log n)$, штангенциркуль $O(h)$, итого $O(n \log n)$.

Важно. Ближайшая пара не сводится к проверке только соседей по x . При Грэме нужно заранее определить политику для коллинеарных точек. В штангенциркуле проверяются антиподальные пары, а не все h^2 пар.

Каркас устного ответа

Ввести cross product. Ближайшая пара: разделение, δ , полоса и не более 7 соседей, рекурсия $O(n \log n)$. Оболочка: сортировка по углу и стек левых поворотов. Диаметр: концы на оболочке, антиподальные пары и два монотонно движущихся указателя.

Сводная таблица сложностей

Задача / алгоритм	Время	Дополнительная память
Карацуба	$\Theta(n^{\log_2 3})$	обычно $O(n)$
Разделение стержня	$\Theta(n^2)$	$\Theta(n)$
Порядок умножения матриц	$\Theta(n^3)$	$\Theta(n^2)$
Обычный LCS	$\Theta(mn)$	$\Theta(mn)$
Хиршберг	$O(mn)$	$O(\min(m, n))$
Insertion sort	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n)$
Heapsort	$O(n \log n)$	$O(1)$
Randomized quicksort	ожид. $O(n \log n)$	ожид. $O(\log n)$ стек
Counting sort	$O(n + k)$	$O(n + k)$
Radix sort	$O(d(n + k))$	зависит от прохода
BFPRT selection	$O(n)$	$O(\log n)$ стек / реализация
BFS, DFS, SCC	$O(V + E)$	$O(V)$
Беллман-Форд	$O(V E)$	$O(V)$
Дейкстра + бинарная куча	$O((V + E) \log V)$	$O(V)$
Варшалл / Флойд-Варшалл	$\Theta(n^3)$	$\Theta(n^2)$
Штрассен	$\Theta(n^{\log_2 7})$	зависит от реализации
Четыре русских	$O(n^3 / \log n)$	до $O(n^2) - O(n^3 / \log n)$
АВЛ / красно-чёрное дерево	$O(\log n)$ на операцию	$O(n)$
В-дерево	$O(\log_t n)$ I/O	$O(n)$
Рабин-Карп	ожид. $O(n + m)$	$O(n)$ или $O(1)$ rolling
KMP	$O(n + m)$	$O(m)$
Ахо-Корасик	$O(S \Sigma + n + z)$	$O(S \Sigma)$
Хаффман	$O(\sigma \log \sigma + n)$	$O(\sigma)$
Суффиксное дерево Укконена	$O(n)$, фикс. алфавит	$O(n)$
Inverse BWT	$O(n + \Sigma)$	$O(n + \Sigma)$
Ближайшая пара	$O(n \log n)$	$O(n)$
Выпуклая оболочка Грэма	$O(n \log n)$	$O(n)$
Диаметр после оболочки	$O(h)$	$O(1)$ сверх оболочки

Как готовить устный ответ

Для каждого билета полезно держать одинаковую структуру ответа:

1. точная постановка и предположения;
2. основная структура данных или состояние динамики;
3. псевдокод на уровне ключевых шагов;
4. один ясный инвариант или лемма корректности;
5. строгая оценка времени и памяти;
6. ограничение применимости: отрицательные веса, вероятностные коллизии, модель сравнений, фиксированный алфавит и т.п.

Источник

Конспект составлен по материалам лекций Александра Охотина «Математические основы алгоритмов», осенний семестр 2025–2026 гг., лекции 1–11. Формулировки переработаны и объединены по экзаменационным билетам; доказательства и анализ сложности развёрнуты до самостоятельного учебного текста.